UCS503 - Software Engineering (Project)

SafeRoute

A Crowdsourced Safety-Aware Navigation Web App

Submitted by:

1024030176 Dhriti Handa
1024030180 Disha Kundra

Group: 3C16 BE Third Year, COE

Submitted to:

Dr. Paramveer Sidhu

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Existing Solutions and the Gap . . . 3
- 1.3 Project Objectives . . . 4
- 1.4 Scope and Deliverables . . . 4

2 Methodology and Design **6**
- 2.1 System Architecture . . . 6
  - 2.1.1 High-Level Design . . . 6
  - 2.1.2 Core Workflow . . . 6
  - 2.1.3 Technical Stack . . . 6
  - 2.1.4 Design Decisions . . . 7
- 2.2 Data Design . . . 7
  - 2.2.1 Seeded Campus Data . . . 7
- 2.3 Implementation Details . . . 8
  - 2.3.1 Module 1: Safety-Weighted Routing . . . 8
  - 2.3.2 Module 2: Secondary Features . . . 9

3 Results and Evaluation **10**
- 3.1 Evaluation Criteria . . . 10
- 3.2 Testing Methodology . . . 10
- 3.3 Performance Metrics . . . 10
  - 3.3.1 Headline Route . . . 10
  - 3.3.2 Route Divergence . . . 11
  - 3.3.3 Effect of a Report . . . 11
  - 3.3.4 Query Time . . . 11
- 3.4 Analysis . . . 11
- 3.5 Risks and Mitigations . . . 12

4 Conclusion **13**
- 4.1 Summary of Work . . . 13
- 4.2 Key Findings . . . 13
- 4.3 Future Work and Recommendations . . . 13
- 4.4 Final Remarks . . . 14

Bibliography **15**

Chapter 1

Introduction

1.1 Background and Context

When we walk home at night, we do not only ask "which road is shortest?" but also "which road feels safe?". Street lighting, foot traffic and past incidents can differ a lot between two roads that take the same time to walk. Today this choice is made by word of mouth, not by any tool.

SafeRoute is an install-free web app that shows two routes between a start and a destination: the *fastest* route and the *safest* route. Safety comes from crowdsourced reports, and the safety score is higher at night.

Guiding principle (time-to-value). Our proposal says to show a working end-to-end loop as early as possible. So the prototype uses manually seeded data and a light technology stack, and we add the heavier parts later.

1.1.1 Problem Statement

Google Maps and Apple Maps optimise only for time, distance and traffic. They give no safety information. The problem has three parts:

- **No safety option:** the shortest path may go through a dark or isolated street and the user cannot know in advance.
- **Safety and SOS apps do not navigate:** Safetipin collects safety data but is a planning tool for city governments; bSafe and Noonlight send SOS alerts but give no route advice.
- **No time-of-day awareness:** a street that is fine at 6 pm can feel very different at 11 pm.

1.2 Existing Solutions and the Gap

Table 1.1 compares existing tools with SafeRoute. SafeRoute is the only one that combines walking routes, safety data and SOS in one install-free web app. (The SafeRoute row is the intended product; Chapter 3 states what the prototype already does.)

Tool	Walking routing	Safety data	SOS	No install	Time of day
Google / Apple Maps	Yes	No	No	Yes	No
Safetipin	No	Yes	No	No	No
bSafe / Noonlight	No	No	Yes	No	No
SafeRoute	Yes	Yes	Yes	Yes	Yes

Table 1.1: Comparison with existing tools

1.3 Project Objectives

1. **Safety-aware routing:** compute a fastest route (distance only) and a safest route (distance weighted by risk).
2. **Crowdsourced reporting:** users submit geotagged reports with an optional photo; each report raises the risk of the nearest road segment.
3. **Time-of-day awareness:** risk is increased at night, so routes and the heatmap change after dark.
4. **Emergency sharing:** an SOS feature creates a live-location link for a saved contact.
5. **Measurable evaluation:** the safest route should differ from the fastest route in at least 80% of test cases that contain a low-safety segment.

1.4 Scope and Deliverables

Table 1.2 shows the status of each deliverable at this mid-semester evaluation.

Deliverable	**Status**	**Note**
Repository and architecture setup	Done	Backend + frontend
Reporting and storage schema	Done	SQLite (not PostGIS)
Fastest and safest routing	Done	Custom Dijkstra, seeded data
Heatmap and night mode	Done	Backend endpoint
SOS quick-share	Partial	SMS is simulated
Web UI (home, report, SOS)	Done	Placeholder data
Frontend connected to API	Pending	Next task
Real map (Mapbox), IDW interpolation	Pending	Planned
Push alerts, CI/CD, load test, deployment	Pending	Later milestones

Table 1.2: Deliverable status at mid-semester

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

SafeRoute has three layers. A React web client talks to a Flask REST API over HTTP/JSON. The API calls two Python modules (one for routing, one for safety) and stores everything in an SQLite database.

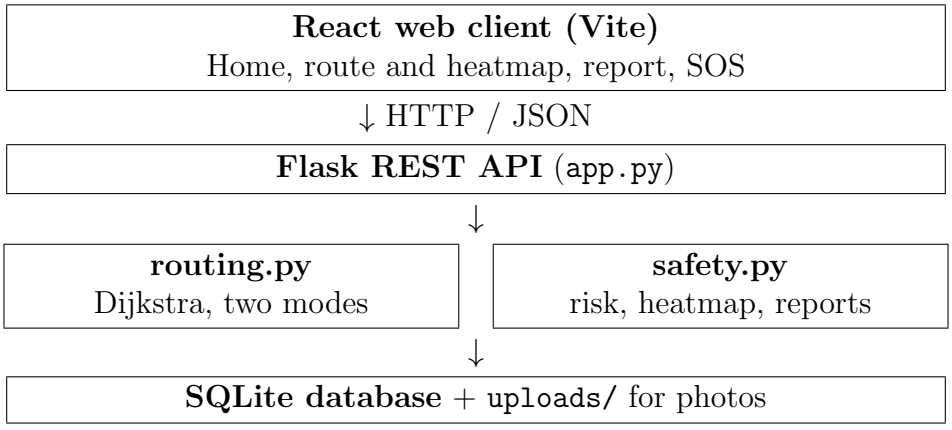


Figure 2.1: Architecture of the SafeRoute prototype

2.1.2 Core Workflow

1. The user picks a start and a destination (`GET /api/places`).
2. The app asks for routes (`GET /api/route`); the backend returns the fastest and the safest route.
3. If the user sees a hazard, they send a report (`POST /api/reports`).
4. The report raises the risk of that road, so the heatmap and the next route request show it immediately.

2.1.3 Technical Stack

Python and Flask [Pal26] for the backend, React [Met26] with Vite [Vit26] for the frontend, and SQLite [SQL26] for the database. Table 2.1 compares this with the

stack planned in the proposal. We chose the lighter stack first so that the full loop could be shown early.

Layer	Prototype (done)	Proposal (planned)
Frontend	React, Vite	+ Mapbox GL JS
Database	SQLite	PostgreSQL + PostGIS
Routing	Own Dijkstra	OSRM [LV11]
Gap filling	None	IDW interpolation [She68]
SOS	Simulated SMS	Real SMS / push
Testing	Manual checks	pytest, Jest, GitHub Actions

Table 2.1: Prototype stack versus planned stack

2.1.4 Design Decisions

- **Explainable scoring:** one simple formula decides the route, so it is easy to test and to explain.
- **Stateless API:** all state is in the database, so the API can be scaled later.
- **Never block the user:** routes are ranked by safety but never hidden; the user chooses.

2.2 Data Design

The database has five tables (Table 2.2). A road segment is an undirected *edge* between two places (*nodes*).

Table	Main columns
`nodes`	code, name, lat, lng
`edges`	id, u, v, name, risk (0 to 1), note
`reports`	id, edge_id, lat, lng, categories, description, photo, created
`contacts`	id, name, phone
`sos`	id, contact_id, lat, lng, active, updated

Table 2.2: Database schema

2.2.1 Seeded Campus Data

`seed.py` fills the database with 8 places around the Thapar campus (G1 = Thapar Gate 1, A = Bus Stand Rd Jn, B = Library Block, C = Model Town Chowk, D =

Station Rd Jn, E = Campus Rd Jn, F = Campus Rd North, BUS = Rajpura Bus Stand) and the 9 road segments in Table 2.3, with hand-assigned risk values. This is the manual seeding the proposal plans for the cold-start phase.

ID	Segment	Road	Risk	Note
1	G1 – A	Bus Stand Rd	0.10	Well-lit main road
2	A – C	Model Town Ln	0.80	Dim stretch
3	C – D	Model Town Ln	0.70	Isolated, no foot traffic
4	D – BUS	Station Rd	0.30	Mixed reports
5	A – B	Bus Stand Rd	0.10	Well-lit, foot traffic
6	B – E	Campus Rd	0.10	Verified safe
7	E – F	Campus Rd	0.15	Verified safe
8	F – BUS	Campus Rd	0.10	Verified safe, shops open
9	B – D	Link Ln	0.50	Poorly lit after 8 pm

Table 2.3: Seeded road segments

2.3 Implementation Details

2.3.1 Module 1: Safety-Weighted Routing

Step 1: night adjustment. At night the stored risk r is multiplied by 1.25 (maximum 1):

$$r_{\text{eff}} = \begin{cases} \min(1,\ 1.25\, r) & \text{night mode} \\ r & \text{otherwise} \end{cases}$$

The risk is then labelled: below 0.3 is *safe*, from 0.3 to 0.6 is *mixed*, and 0.6 or more is *risk*.

Step 2: cost of a segment. With d the length of the segment in metres (haversine formula on the coordinates):

$$w = \begin{cases} d & \text{fastest route} \\ d\,(1 + 4\, r_{\text{eff}}) & \text{safest route} \end{cases}$$

A fully risky segment costs five times its length, so the safest route accepts a small detour to avoid it; a segment with no risk costs exactly its length.

Step 3: shortest path. Dijkstra's algorithm [Dij59] (with a binary heap) runs twice on the road graph, once with each cost. The API returns, for each route, the coordinates, distance, walking time (83 m/min), average risk and turn-by-turn steps, plus a `diverges` flag that says whether the two routes differ.

2.3.2 Module 2: Secondary Features

REST API. Table 2.4 lists the endpoints.

Method	Endpoint	Purpose
GET	`/api/places`	List places
GET	`/api/route`	Fastest and safest route
GET	`/api/heatmap`	Risk of every segment
POST	`/api/reports`	Submit a report (+ photo)
GET	`/api/reports`	Latest 20 reports
GET/POST	`/api/contacts`	List or add contacts
POST	`/api/sos`	Start an SOS session
GET/PUT/DELETE	`/api/sos/<id>`	Read, update or end session

Table 2.4: Backend REST endpoints

Incident reporting. A report has a location, categories, a description and an optional photo (saved with a random filename). The backend finds the nearest road segment, raises its risk by 0.15 (maximum 1) and updates its note. Since routing reads the same table, the next route request already uses the new risk.

Heatmap. `/api/heatmap` returns every segment with its effective risk; with `night=1` the same map looks riskier.

SOS quick-share. Starting SOS creates an 8-character session ID for a saved contact and stores the user's location. The location can be updated and the session ended later. In the prototype the SMS is simulated: the tracking link is printed in the server console.

Frontend. The React app has three screens: *Home* (places, fastest/safest toggle, route and heatmap tabs), *Report* (issue types and details) and *SOS* (hold button, share location, stand down). Currently they use placeholder data and a drawn map; connecting them to the API is the main remaining task.

Running the prototype. `python app.py` in `code/backend` (the database is seeded automatically) and `npm install && npm run dev` in `code/frontend`.

Chapter 3

Results and Evaluation

3.1 Evaluation Criteria

Our main metric is **route divergence**: the safest route should differ from the fastest in at least 80% of test cases where the fastest route has a risky segment. Secondary metrics are listed in Table 3.1.

Metric	Target	Status
Route divergence	$\geq 80\%$	Measured: 43%
Report-to-heatmap time	< 10 s	Measured: a few ms (in-process)
Interpolation check, load test, uptime	–	Pending

Table 3.1: Evaluation criteria and status

3.2 Testing Methodology

- **Unit testing:** routing and safety functions were run directly in Python on the seeded data; automated `pytest` tests are planned.
- **Integration testing:** `/api/places` and `/api/route` were called in a browser and returned the expected data. Report, contact and SOS endpoints are not yet tested end to end.
- **Divergence testing:** the router was run for all $8 \times 7 = 56$ ordered pairs of places in night mode.
- **Report test:** one report was added and routes were recomputed.
- **Performance and user testing:** timing was measured in-process only; load test and user pilot come later.

3.3 Performance Metrics

3.3.1 Headline Route

For Thapar Gate 1 to Rajpura Bus Stand, the fastest route uses Link Lane (poorly lit) and Station Road (mixed). The safest route stays on Campus Road. Table 3.2

shows the result.

Setting	Route	Distance (m)	Time (min)	Avg. risk
Day	Fastest	976	12	0.26
Day	Safest	1030	12	0.11
Night	Fastest	976	12	0.32
Night	Safest	1030	12	0.14

Table 3.2: Fastest versus safest route, G1 to BUS

The safest route is only 54 m (about 5.5%) longer but cuts the average risk by more than half (58% by day, 56% at night).

3.3.2 Route Divergence

A pair of places is a test case if its fastest route has at least one *risk* segment. There were 28 such cases, and the routes diverged in 12 of them, i.e. **43%**, below the 80% target.

3.3.3 Effect of a Report

A "Poor lighting" report on Link Lane raised its risk from 0.50 to 0.65. The fastest route's night risk rose from 0.32 to 0.36, while the safest route stayed at 0.14 because it already avoids Link Lane.

3.3.4 Query Time

Computing both routes took about 0.13 ms on average over 500 runs. This is in-process on a tiny graph (8 places), so it is not a substitute for the planned load test.

3.4 Analysis

- **Did we meet the objectives?** Safety-aware routing, reporting, night mode and SOS work in the backend. The 80% divergence target was *not* met (43%).
- **Why is divergence low?** Of the 16 cases that did not diverge, 8 involve Model Town Chowk (C), which is connected only through two risky roads, so no safer path exists. The other 8 involve Station Rd Jn (D): the safer path exists but is much longer, so the cost still prefers the direct road. Raising the risk weight from 4 to 8, 12 and 20 did not change the result, so the limit comes from the small map, not the formula.

- **What worked well?** The cost function is simple, one algorithm serves both modes, and reports and night mode change the output immediately.
- **Challenges:** PostGIS and OSRM were too heavy for the first iteration, so we built the cost function over a small graph. Frontend and backend were built in parallel, so integration is left for the next phase.
- **Known gaps:** frontend not connected to the API; map is a drawing; a report adds a fixed +0.15 with no upvotes; any request can raise a risk (no protection against fake reports).

3.5 Risks and Mitigations

Risk	**Mitigation**	**Status**
No data for new areas (cold start)	Seed scores; interpolate later	Seeded; interpolation pending
Fake reports	Rate limit, minimum upvotes	Not implemented
OSRM complexity	Own router first	Avoided for now
Slow queries at scale	Spatial indexes, load test	Pending

Table 3.3: Main risks from the proposal

Chapter 4

Conclusion

4.1 Summary of Work

We built a Flask backend with a seeded campus graph, a safety-weighted Dijkstra router (fastest and safest), night-time risk adjustment, a heatmap, photo-supported reporting that updates road risk, and a simulated SOS flow. We also built the React screens. The routing, reporting, time-of-day and SOS objectives work in the backend; the divergence target needs a larger map.

4.2 Key Findings

- A simple cost $d(1 + 4r)$ steers routes away from risky roads: a 54 m detour halved the average risk on the headline route.
- A report affects the very next route request, because routing and reporting share one table.
- Divergence depends on map structure: only 43% on the small seeded graph, since it has few parallel streets.

4.3 Future Work and Recommendations

1. Connect the React screens to the API and add a real map.
2. Move to PostgreSQL with PostGIS; compare with OSRM routing.
3. Add IDW interpolation [She68] for areas without reports, and weight reports by recency and upvotes.
4. Add login, rate limiting and automated tests (`pytest`, GitHub Actions).
5. Use a larger or real street map to retest the 80% target.
6. Send real SOS messages, run a pilot with 10–20 peers, load test and deploy.

4.4 Final Remarks

The prototype shows that making safety a routing input is feasible with modest tools. The main lesson is to get one complete loop working early and let honest measurements, including the missed target, decide what to improve next.

Bibliography

[Dij59] Edsger W. Dijkstra. “A Note on Two Problems in Connexion with Graphs”. In: *Numerische Mathematik* 1.1 (1959), pp. 269–271.

[LV11] Dennis Luxen and Christian Vetter. “Real-Time Routing with OpenStreetMap Data”. In: *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. 2011, pp. 513–516.

[Met26] Meta Open Source. *React Documentation*. `https://react.dev`. 2026.

[Pal26] Pallets Projects. *Flask Documentation*. `https://flask.palletsprojects.com`. 2026.

[She68] Donald Shepard. “A Two-Dimensional Interpolation Function for Irregularly-Spaced Data”. In: *Proceedings of the 23rd ACM National Conference*. 1968, pp. 517–524.

[SQL26] SQLite Consortium. *SQLite Documentation*. `https://www.sqlite.org/docs.html`. 2026.

[Vit26] Vite Contributors. *Vite Documentation*. `https://vite.dev`. 2026.